CHEATSHEET PAO

Rezolvare subiect IV - clasa mutabila/imutabila

CLASA IMUTABILA = CLASA CARE ESTE DECLARATA CA “PUBLIC **FINAL**”, ARE ATRIBUTE “PRIVATE **FINAL**“ SI NU ARE SETTERI!

exemplu:

IV. Se consideră definită complet clasa mutabilă `Adresa` care permite memorarea unei adrese din România: *stradă*, *număr*, *bloc*, *scară*, *etaj*, *apartament*, *județ* și *localitate*. Definiți complet o clasă imutabilă `Facultate` care să permită memorarea următoarelor informații despre o facultate: *denumirea facultății* (șir de caractere), *numărul de studenți* (număr natural), *specializări* (listă cu elemente de tip `String`) și *adresa* (referință spre un obiect de tip `Adresa`).

Model de rezolvare - gasesti in Curs 4 PAO Boriga

Exemplu: Fie o clasă `Student` care conține data membră `facultate` de tipul unei clase mutabile `Facultate`:

Greșit:

```
public Student(Facultate f, …){
  this.facultate = f;              //agregare, deci obiectul
  ……                               //extern poate fi modificat!
}
```

Corect:

```
public Persoana(Date dn, …){
  this.facultate = new Facultate(f);    //compoziție, deci obiectul
  ……                                    //extern nu poate fi modificat
}
```

b. nu se vor returna referințe spre câmpurile mutabile, ci se vor returna referințe spre copii ale lor:

Greșit:

```
public Facultate getFacultate(){
  return this.facultate;
}
```

Corect:

```
public Facultate getFacultate(){
  return new Facultate(facultate);
}
```

- În continuare, vom prezenta complet clasele `Facultate` și `Student` menționate anterior:

Clasa mutabilă `Facultate`:

```
public class Facultate {
    private String denumire;
    private String adresa;
    private String email;
    private String telefon;

    public Facultate(String denumire, String adresa, String email,
                                                      String telefon) {
        this.denumire = denumire;
        this.adresa = adresa;
        this.email = email;
        this.telefon = telefon;
    }
```

```
    public Facultate(Facultate facultate) {
        this.denumire = facultate.denumire;
        this.adresa = facultate.adresa;
        this.email = facultate.email;
        this.telefon = facultate.telefon;
    }

    public String getDenumire() { return denumire; }

    public void setDenumire(String denumire) { this.denumire = denumire; }

    public String getAdresa() { return adresa; }

    public void setAdresa(String adresa) { this.adresa = adresa; }

    public String getEmail() { return email; }

    public void setEmail(String email) { this.email = email; }

    public String getTelefon() { return telefon; }

    public void setTelefon(String telefon) { this.telefon = telefon; }
}
```

Clasa imutabilă Student:

```
//regula 1
public final class Student
{
    //regula 2
    private final String nume;
    private final Facultate facultate;
    private final int grupa;
    private final double medie;

    public Student(String nume, Facultate facultate, int grupa, double medie) {
        this.nume = nume;
        //regula 4a
        this.facultate = new Facultate(facultate);
        this.grupa = grupa;
        this.medie = medie;
    }

    public String getNume() {
        return nume;
    }
```

```
    public Facultate getFacultate() {
        //regula 4b
        return new Facultate(facultate);
    }

    public int getGrupa() {
        return grupa;
    }

    public double getMedie() {
        return medie;
    }
}
```

Observați faptul că în clasa `Student` nu sunt definite metode de tip `set`, deci este respectată și regula 3!

Rezolvare subiect IV - JDBC

IV. Se consideră baza de date *Angajati*, având următorul URL: *jdbc:derby://localhost:1527/Angajati* . Baza de date conține tabela *DateAngajati*, având câmpurile *CNP*, *Nume*, *Varsta* și *Salariu*. Scrieți un program care să citească de la tastatură o valoare reală *s* și valoare întreagă *v*, după care afișează informațiile despre angajații având cel mult *v* ani și salariul cel puțin *s* RON.

Model de rezolvare - Pepi

```
package Subiect_IV;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Scanner;

// BORIGA NU TI VA CERE NEVER SA PUI IMPORT URI CAND SCRII

public class Subiect_IV {
```

```
	public static void main(String[] args) {
// Lucrezi doar in main, acolo unde vei incepe sa configurezi conexiunea la baza de date (URL, USER, PASSWPRD)

	try (Connection conn = DriverManager.getConnection("jdbc:mysql://sql7.freemysqlhosting.net:3306/sql7620469" , "sql7620469" , "ATZWkKKaN5");) {

		Scanner in = new Scanner(System.in); // citire informatii

		System.out.println("Salariul minim: ");
		float s = in.nextFloat();

		System.out.println("Varsta maxima: ");
		int v = in.nextInt();

		PreparedStatement pst = null;
		ResultSet rs = null;

		pst = conn.prepareStatement("SELECT * FROM angajati WHERE Salariu > ? AND Varsta < ?"); // afisare in display - script SELECT * FROM
		pst.setFloat(1, s);
		pst.setInt(2, v);

		rs = pst.executeQuery();

		while (rs.next()) {
		System.out.println(rs.getString("Nume") + " " + rs.getFloat("Salariu") + " " + rs.getInt("Varsta"));
		}

	} catch (SQLException ex) { // exceptie eroare JDBC
		System.out.println("Eroare JDBC: " + ex);
	}
	}
}
```

Rezolvare subiect II - STREAM-URI

La stream-uri treaba sta cam in felul urmator:

- Partea asta de scriere a stream-urilor si a functiilor lambda este fff asemanator cu Haskell-ul, deoarece partea asta tine de Programare Functionala.
- Este destul de intuitiv: Fiecare subpunct are mereu cam acelasi tip de rezolvare (mereu se aplica anumite tipuri de functii). Pentru acest tip de exercitiu, este suficient sa stii cum sa folosesti si sa scrii functiile: .filter(), .sorted(), .map(), .mapTo...(), .forEach(), collect(), equals(), max() etc. (in principiu, cam acestea). Exercitiile de genul sunt repetitive, mereu iti va cere aceleasi chestii, dar cu variabile diferite :))))))).

Fie acest exercitiu cu stream-uri dintr- un model de examen de anul trecut:

II. Se consideră definită o clasă `Produs` având datele membre `firma`, `produs`, `cantitate` şi `pret_unitar`. Clasa este utilizată pentru a memora informaţii despre produsele existente într-un magazin. Datele membre `firma` şi `produs` sunt de tip `String`, iar `cantitate` şi `pret_unitar` sunt de tip `float`. Clasa încapsulează metode de tip `set/get` pentru toate datele membre, precum şi metodele `toString()`, `equals()` şi `hashCode()`. Creaţi o lista care să conţină cel puţin 3 obiecte de tip `Produs` şi, folosind stream-uri bazate pe lista creată şi lambda expresii, rezolvaţi următoarele cerinţe:

- afişaţi produsele al căror preţ unitar este cel puţin 100 RON, în ordinea descrescătoare a cantităţilor;
- afişaţi firmele producătoare distincte, în ordine alfabetică;
- creaţi o colecţie care să conţină produsele având valoarea totală cel mult 1000 RON;
- afişaţi preţul unitar mediu al tuturor produselor.

- Primul pas: NU E NECESAR SA DEFINESTI CLASA DATA, DOAR PIERZI TIMP (daca te ajuta sa vezi mai bine anumite lucruri, faci SUMAR ceva schematic pe o ciorna)
- Implementarea prorpiu-zisa o incepi din MAIN!!!
- Vei incepe sa ti creezi o lista cu obiecte de tip Produs sub aceasta forma:

```
public static void main(String[] args) {
```

```
List<Produs> produse = new ArrayList<>();
produse.add(new Produs(date_membru));
produse.add(new Produs(date_membru));
produse.add(new Produs(date_membru));
// mai poti adauga daca vrei
```

- Pentru fiecare subpunct, trebuie mereu sa incepi cu initializarea stream-ului, in functie de ceea ce-ti cere enuntul.

- afișați produsele al căror preț unitar este cel puțin 100 RON, în ordinea descrescătoare a cantităților;

  - PAS 1: Initializam stream-ul. In acest caz, iti cere sa afisezi PRODUSE, adica obiecte de tip PRODUS, vei initializa stream-ul astfel:
    `List<Produs> produseDesc = produse.stream()` // poti pune ce denumire vrei tu in loc de produseDesc, in functie de ceea ce ti cere euntul

  - PAS 2: INCEPEM APLICAREA FUNCTIILOR. Citind enuntul, deducem ca ceea ce avem de facut este sa FILTRAM produsele dupa UN ANUMIT PRET si sa SORTAM produsele descrescator => vom aplica .filter() si .sorted()
    In continuarea exercitiului vei scrie asa:
    `.filter(p ->p.getPretUnitar() >= 100)`
    `.sorted(Comparator.comparingDouble(Produs::getPretUnitar()).reversed())`

    Luand pe rand: avem un p (de la produs) pe care il filtram in functie de PRETUL UNITAR => ne folosim de getter-ul pe care il vom presupune ca fiind implementat in clasa PRODUS si vom spune ca filtram sau mai bine zis SELECTAM produsele cu pretul cel putin de 100 de LEI, dupa aceea le sortam DESCRESCATOR (.reversed()), iar in cadrul functiei de sortare, cel mai simplu, putem folosi clasa Comparator si de functia de

comparing, unde selectam din clasa Produs preturile unitare prin getter).

- PAS 3: Afisati PRODUSELE (mai multe) -> afisare lista -> folosim mereu .collect() si .forEach()
  .collect() -> populeaza colectia/ lista ta pe care tu o dai ca output
  Ca sintaxa, vei avea mereu => .collect(Collectors.toList()) <= ASA VEI SCRIE MEREU

  .forEach() -> iti afiseaza mereu fiecare element din lista
  Ca sintaxa, vei avea mereu => .forEach(System.out::println)

  Ideea e ca mereu cand ai de afisat mai multe elemente dintr-o lista, cam dai copy paste ultimelor 2 randuri

  Rationamentul este acelasi pentru toate subpunctele: Cand iti cere sa afisezi niste obiecte mai particulare, folosesti FILTER, cand iti zice de ordine folosesti SORTED, cand afisezi elemente dintr o lista, folosesti COLLECT si FOREACH.

  Cand ai, de exemplu:

  - afişaţi firmele producătoare distincte, în ordine alfabetică;

  Firmele producatoate DISTINCTE => vei afisa o lista de String-uri! Iar pentru a afisa elemente distincte, ne vom folosi de .MAP() si .DISTINCT(). Dupa aceea, la fel, afisarea elemetelor o vei face cu .COLLECT() si cu .FOREACH(). Ordine alfabetica => .SORTED()

  - creaţi o colecţie care să conţină produsele având valoarea totală cel mult 1000 RON;

  Colectie de produse => LISTA CU OBIECTE DE TIP PRODUS
  Produse ce contin ANUMITE valori => .FILTER()

Afisare produse => .COLLECT() + .FOREACH()

- afișați prețul unitar mediu al tuturor produselor.
  - In acest caz, nu mai ai de afisat MAI MULTE lucruri, ci doar O VALOARE => declari stream-ul cam ceva de genul:
    double pretUnitarMediu = produse.stream()
    Afisarea o vei face doar cu System.out.println()

Si cam acesta este rationamentul pentru rezolvarea acestui subiect

Rezolvare subiect III - THREAD-URI

Aici ce conteaza e sa stii sa faci citirea din fisier si sa II DECLARI COMPLET CLASA (date_membru + constructor + getteri/setteri), dupa te apuci de clasa in care faci operatiile pentru “firul tau executabil” => citirea din fisier
Las aici un model de implementare:

THREAD-URI

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
import java.util.SplittableRandom;
public class FirExecutor extends Thread{ // declaru clasa
    private String numeFisier; // parametrii cu numele fisierului
    private String cuvant;
    private int contor;
public FirExecutor ( String numeFisier, String cuvant, int contor){
        this.numeFisier = numeFisier;        // constructor
        this.cuvant = cuvant;
        this.contor = contor;
public int getContor(){
    return contor;
                }

@Override
public void run(){ // metoda run() care se ocupă de thread
    Scanner in=null;
    try {
        in = new Scanner (new File (numeFisier));
    while(in.hasNextLine()){
            String linie = in.nextLine();
            String cuvinte[] = linie.split("[\\A,.:!?\n+-]+");
            for (int i=0; i< cuvinte.length; i++){
                if (cuvinte[i].equals (cuvant)){
                    contor++;
                }
            }
        }
    } catch (FileNotFoundException e){
            System.err.println ("Fisierul nu a fost găsit: "+numeFisier);
    } finally{
        if(in!=null){
            in.close();
    }
}
}
```

```
Scanner in = new Scanner(System.in)
System.out.println("Cuvântul căutat: ");
String cuvânt = in.nextLine()
```

Citire de la tastatură

```
FirExecutator fir1 = new FirExecutator("src/Exemple_1.txt",
                                       cuvânt, 0);
fir2 = ~
fir3 = ~
```

Creăm câte un fir de execuție

```
fir1.start();
fir1.join();
fir2.start();
fir2.join();
~
~
```

fir.start() + fir.join

```
int suma = fir1.getContor() + fir2.getContor() + fir3 ~
System.out.println(suma);
```

// Accesăm getterii

Ce e marcat cu galben e cam cel mai relevant lucru, parerea mea

GRILE